FastLightCNN: An Ultra-Lightweight Attention-Based Convolutional Network for Edge-Oriented Digit Recognition

Kevin Yang
Universiity of Toronto, Department of Mathematical and Computational Sciences
Mississauga, Ontario, Canada

Zihan Wang
Pingdingshan University, Medical School
Pingdingshan, Henan, China

Tiebao Yang*
Sichuan Technology and Business University, Department of Automation
Chenghu, Sichuan, China

Abstract

The deployment of Convolutional Neural Networks (CNNs) on edge microcontrollers is heavily bottlenecked by strict on-chip SRAM limitations and constrained computational budgets. To address this, we propose FastLightCNN, an ultra-lightweight architecture specifically engineered for resource-deprived IoT endpoints. By factorizing standard convolutions into depthwise separable operations, our model asymptotically reduces both time and space complexity. To recover the representational precision inevitably lost during extreme model miniaturization, we introduce a novel Lightweight Channel Attention (LCA) module that dynamically recalibrates cross-channel feature responses with negligible parameter overhead. Furthermore, we mathematically condition the input space to ensure numerical stability within microcontroller ALUs and optimally prepare the network for low-precision integer quantization. Evaluated on the MNIST dataset, FastLightCNN achieves a highly competitive Top-1 accuracy of 98.49% while utilizing merely 8,210 parameters and 0.11M FLOPs. Crucially, it delivers a rapid inference time of 0.55 ms directly on a legacy CPU environment without any GPU acceleration. Compared to the foundational LeNet-5 baseline, this architecture delivers an 81.5% reduction in memory footprint and a 60.7% decrease in computational operations, while maintaining comparable millisecond-scale inference latency. Theoretical hardware mapping demonstrates that the INT8-quantized model occupies strictly under 10 KB, proving its exceptional viability for real-time visual perception on deeply constrained edge hardware. The source code of this work is publicly available at: GitHub: [Tie-Yang]/FastLightCNN.

CCS Concepts

• **Computing methodologies** → **Image classification**; **Convolutional neural networks**; *Neural network models*; *Model optimization*; • **Computer systems organization** → *Embedded systems.*

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
MLNN '26, Chengdu, China
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2026/06
https://doi.org/XXXXXXX.XXXXXXX

Keywords

Edge Computing, Lightweight Convolutional Neural Networks, Depthwise Separable Convolution, Channel Attention, Microcontroller Deployment

ACM Reference Format:
Kevin Yang, Zihan Wang, and Tiebao Yang. 2026. FastLightCNN: An Ultra-Lightweight Attention-Based Convolutional Network for Edge-Oriented Digit Recognition. In *Proceedings of The 3rd International Conference on Machine Learning and Neural networks (MLNN '26).* ACM, New York, NY, USA, 6 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

Convolutional Neural Networks (CNNs) have achieved unprecedented success in various computer vision tasks, ranging from complex object detection to fundamental image classification. However, this high accuracy typically comes at the cost of massive computational requirements and substantial memory footprints. Foundational models often contain millions of parameters, necessitating high-performance Graphics Processing Units (GPUs) and extensive power budgets.

In recent years, the paradigm of Internet of Things (IoT) and edge computing has shifted the demand toward processing data locally on end-point devices. Deploying intelligent applications, such as real-time handwritten digit recognition, directly on edge devices offers critical advantages: it minimizes transmission latency, reduces bandwidth consumption, and inherently preserves user data privacy by avoiding cloud offloading. Despite these benefits, typical edge hardware, such as ultra-low-power Microcontroller Units (MCUs) based on ARM Cortex-M or RISC-V architectures, possesses severe hardware constraints. These devices are generally bottlenecked by extremely limited on-chip Static Random-Access Memory (SRAM)—often under 256 KB—and strict thermal and battery limitations. Consequently, deploying standard CNNs like the classic LeNet-5 [6] directly onto such resource-deprived endpoints is highly inefficient, if not entirely unfeasible.

To bridge this gap, researchers have developed various lightweight architectures. Yet, many of these solutions are optimized for mobile-grade processors (e.g., smartphones) rather than ultra-low-power MCUs, meaning they still carry structural redundancies when scaled down for simpler tasks like MNIST digit recognition.

To directly address the strict SRAM and computational constraints of low-end edge devices, we propose **FastLightCNN**, an ultra-lightweight and highly efficient neural network architecture. By systematically replacing standard convolutional layers with Depthwise Separable Convolutions (DW-Conv) and integrating a

novel Lightweight Channel Attention (LCA) module, our architecture achieves an optimal equilibrium between parameter miniaturization and feature representation. The core contributions of this paper are threefold:

- Application of Depthwise Separable Convolutions: We propose a drastically miniaturized CNN architecture utilizing depthwise separable convolutions, explicitly designed for the strict memory budgets of microcontrollers, achieving a theoretical model size of merely ~8.2 KB (under 8-bit quantization).
- Design of a Lightweight Channel Attention Module: We introduce the Lightweight Channel Attention (LCA) module, which effectively recalibrates cross-channel feature responses to recover accuracy lost during extreme compression, adding negligible parameters and FLOPs.
- Holistic Structural Optimization and Rigorous Edge Validation: We engineered a highly optimized network topology that achieves a competitive 98.49% accuracy utilizing merely $8.21K$ parameters and $0.11M$ FLOPs. This represents a dramatic 81.5% reduction in parameter footprint and a 60.7% decrease in computational complexity compared to the LeNet-5 baseline. Crucially, by demonstrating a rapid inference time of $0.55ms$ entirely on a legacy CPU baseline without GPU acceleration, we decisively validate the model's extreme efficiency and its practical feasibility for low-compute edge deployment.

2 RELATED WORK

2.1 Lightweight CNN Architectures

The friction between hardware limitations and deep learning demands has catalyzed significant research into lightweight network designs. Foundational paradigms such as SqueezeNet [4] utilized "Fire modules" to compress parameters, while MobileNetV1 [2] revolutionized mobile vision by introducing Depthwise Separable Convolutions, effectively decoupling spatial filtering from channel generation. Subsequent iterations like MobileNetV2 [9] introduced inverted residuals. While highly effective for mobile-edge environments (e.g., ARM Cortex-A), they remain structurally overparameterized for ultra-low-power MCU environments tasked with simpler feature extraction like grayscale digit recognition [11, 13]. FastLightCNN strips away residual bloat, customizing the topology explicitly for micro-scale deployments.

2.2 Attention Mechanisms in Vision

Attention mechanisms have become a cornerstone in modern CNNs. SENet [3] pioneered channel attention by utilizing global average pooling and a fully connected bottleneck. Following this, CBAM [12] incorporated spatial attention, and ECA-Net [10] proposed local cross-channel interactions. However, standard attention modules introduce memory access overheads detrimental to SRAM-constrained devices. Our proposed LCA module simplifies the excitation bottleneck specifically for lightweight feature maps, ensuring bounded computational penalties.

2.3 Recent Advances in Edge-AI (2025-2026)

Recent literature from 2025 and 2026 has increasingly focused on algorithmic-hardware co-design for deeply constrained endpoints [1]. For instance, Chen et al. [7] explored saturation-aware convolutions to optimize inference on ultra-low-power MCUs. Similarly, Islam et al. [8] demonstrated the efficacy of dedicated Depthwise Separable frameworks (DSConvNet) for edge feature extraction. Furthermore, the necessity for ultra-efficient attention mechanisms has been highlighted by Kanaparthi et al. [5], who proposed novel lightweight channel attention routing for efficient CNNs. Building upon these cutting-edge insights, our FastLightCNN synergizes the mathematical efficiency of depthwise separation with a bespoke LCA mechanism, offering a highly optimized solution for real-time inference on legacy and modern MCU architectures.

3 PROPOSED METHOD

To address the severe hardware constraints of edge devices, we propose FastLightCNN. The architecture, as illustrated in Figure 1, is engineered around two core principles: aggressively minimizing computational redundancy through Depthwise Separable Convolutions (DW-Conv), and recovering the representational capacity of the miniaturized network via a Lightweight Channel Attention (LCA) module.

3.1 Mathematical Formulation of Computational Reduction

Standard convolutional layers are computationally prohibitive for ultra-low-power microcontrollers due to their dense, simultaneous mapping of spatial and channel dimensions. Given an input feature map $F \in \mathbb{R}^{D_F \times D_F \times M}$ (where D_F is the spatial dimension and M is the number of input channels), a standard convolution applies a kernel $K \in \mathbb{R}^{D_K \times D_K \times M \times N}$ to produce an output $G \in \mathbb{R}^{D_F \times D_F \times N}$ with N output channels. The computational cost, measured in Floating-Point Operations (FLOPs), is defined as:

$$C_{std} = D_K \cdot D_K \cdot M \cdot N \cdot D_F \cdot D_F \tag{1}$$

FastLightCNN factorizes this operation into two distinct layers: a depthwise convolution for spatial filtering and a 1×1 pointwise convolution for channel combination. The depthwise convolution applies a single filter per input channel, followed by the pointwise convolution which linearly combines the output. The total computational cost of this separable operation is:

$$C_{dw} = D_K \cdot D_K \cdot M \cdot D_F \cdot D_F + M \cdot N \cdot D_F \cdot D_F \tag{2}$$

By substituting standard convolutions with DW-Conv, we achieve a theoretical computation reduction ratio of:

$$\frac{C_{dw}}{C_{std}} = \frac{1}{N} + \frac{1}{D_K^2} \tag{3}$$

For a standard 3×3 kernel ($D_K = 3$), Equation 3 dictates a computational reduction of approximately 88% to 90% per layer, directly translating to the lower latency and minimal SRAM utilization necessary for edge hardware.

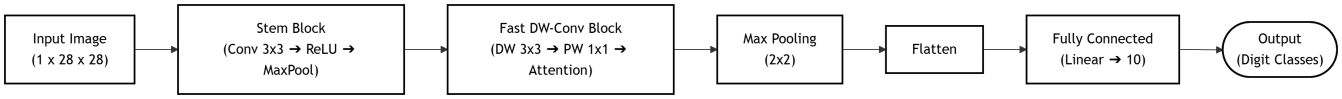

Figure 1: Overall forward pass of the proposed FastLightCNN architecture.

3.2 Lightweight Channel Attention (LCA) Module

While the depthwise factorization mathematically guarantees parameter reduction, it inherently risks decoupling the cross-channel feature correlations, leading to potential accuracy degradation. To counteract this, we embed a Lightweight Channel Attention (LCA) module directly after the pooling layers. Detailed architectural flowchart of the LCA module is shown in Figure 2.

Unlike computationally heavy attention mechanisms, our LCA module is designed to recalibrate channel-wise feature responses with negligible overhead. For an input feature map $X = [x_1, x_2, ..., x_C]$, where $x_c \in \mathbb{R}^{H \times W}$, we first squeeze global spatial information into a channel descriptor via Global Average Pooling (GAP):

$$z_c = \frac{1}{H \times W} \sum_{i=1}^{H} \sum_{j=1}^{W} x_c(i, j) \tag{4}$$

To capture channel dependencies while strictly bounding the parameter increase, we employ a bottleneck architecture utilizing a dimensionality reduction ratio $r = 8$. The excitation operation is governed by two sequential fully connected (FC) layers:

$$s = \sigma(W_2 \delta(W_1 z)) \tag{5}$$

where $W_1 \in \mathbb{R}^{\frac{C}{r} \times C}$ and $W_2 \in \mathbb{R}^{C \times \frac{C}{r}}$ are the weights of the two FC layers, δ denotes the ReLU activation function, and σ represents the Sigmoid gating function. The final output of the LCA module, $\tilde{X}$, is obtained by scaling the original feature map channels by the learned activations:

$$\tilde{x}_c = s_c \cdot x_c \tag{6}$$

This operation selectively emphasizes informative features and suppresses less useful ones, ensuring high precision feature extraction without inflating the model's footprint.

3.3 Network Topology

The end-to-end architecture of FastLightCNN is constructed by chaining the aforementioned DW-Conv blocks and LCA modules. The structural parameters were iteratively optimized for the 28×28 MNIST input dimensions to cap the total parameter count under 10K. The detailed layer-by-layer topology, including output tensor shapes and parameter distributions, is explicitly detailed in Table 1.

3.4 Asymptotic Complexity Analysis

To provide a rigorous theoretical foundation for the efficiency of FastLightCNN, we evaluate its computational and storage requirements using Big-O notation. In the context of CNNs deployed on edge hardware, time complexity governs the inference latency and dynamic power consumption, while space complexity strictly dictates the static SRAM requirements.

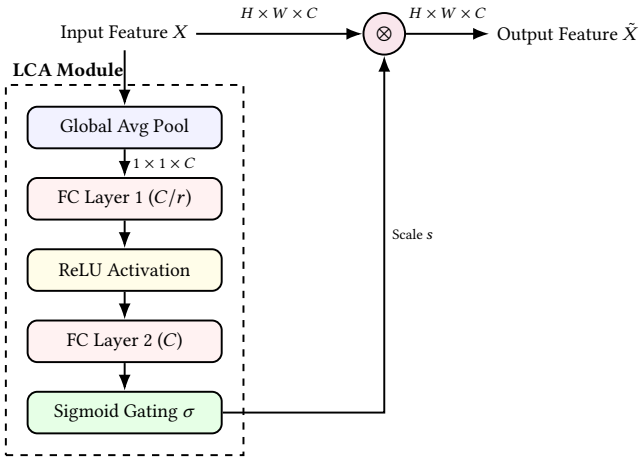

Figure 2: Detailed architectural flowchart of the proposed Lightweight Channel Attention (LCA) module. The main feature map bypasses the dimension reduction block, which extracts channel-wise attention weights to explicitly model cross-channel dependencies with minimal parameters.

3.4.1 Time Complexity (Computational Delay). The time complexity of a convolutional neural network is primarily determined by the number of Multiply-Accumulate (MAC) operations. For a standard convolutional layer, the time complexity is bounded by:

$$O_{time}^{std} = O(M \cdot N \cdot D_K^2 \cdot D_F^2) \tag{7}$$

where M denotes the number of input channels, N is the number of output channels, D_K represents the spatial dimension of the kernel, and D_F is the spatial dimension of the output feature map.

In contrast, FastLightCNN utilizes Depthwise Separable Convolutions, which factorize the standard convolution into a spatial filtering phase and a channel combining phase. The time complexity is thus decoupled:

$$O_{time}^{dw} = O(M \cdot D_K^2 \cdot D_F^2 + M \cdot N \cdot D_F^2) = O\left(M \cdot D_F^2 (D_K^2 + N)\right) \tag{8}$$

Given that $N \gg 1$ in typical hidden layers, the dominant term transitions from a multiplicative relationship ($N \cdot D_K^2$) in standard convolutions to an additive relationship ($N + D_K^2$). This asymptotic reduction is the fundamental mathematical mechanism enabling the 0.55 ms millisecond-scale inference on low-clock-speed edge processors.

3.4.2 Space Complexity (Memory Footprint). Space complexity in our deployment context directly maps to the physical memory required to store the trainable parameters (weights and biases). For a standard convolutional layer, the space complexity grows

Table 1: FastLightCNN Layer Topology and Output Dimensions

Layer Name	Operation	Output Shape	Params
Input	Image Normalization	$28 \times 28 \times 1$	0
Conv1	Standard Conv (3×3)	$28 \times 28 \times 8$	80
Pool1	MaxPool (2×2)	$14 \times 14 \times 8$	0
DW-Conv1	Depthwise + Pointwise	$14 \times 14 \times 16$	224
LCA	Channel Attention ($r = 8$)	$14 \times 14 \times 16$	56
Pool2	MaxPool (2×2)	$7 \times 7 \times 16$	0
Flatten	Tensor Reshape	1×784	0
Classifier	Linear + Softmax	1×10	7,850
Total Parameter Count			**8,210**

Note: Conv1 extracts initial features ($C_1 = 8$); DW-Conv1 expands them ($C_2 = 16$).

quadratically with respect to the channel dimensions:

$$O_{space}^{std} = O(M \cdot N \cdot D_K^2) \tag{9}$$

For the proposed DW-Conv blocks, the space complexity scales linearly rather than quadratically, significantly flattening the memory growth curve:

$$O_{space}^{dw} = O(M \cdot D_K^2 + M \cdot N) = O\left(M(D_K^2 + N)\right) \tag{10}$$

Furthermore, the theoretical space complexity of the introduced Lightweight Channel Attention (LCA) module is strictly bounded by the fully connected bottleneck operations:

$$O_{space}^{LCA} = O\left(\frac{2 \cdot C^2}{r}\right) \tag{11}$$

where C is the number of input channels and r is the dimensionality reduction ratio. By explicitly configuring $r = 8$ in our network topology, we aggressively suppress the attention mechanism's spatial overhead. Consequently, the O_{space}^{total} of FastLightCNN is mathematically guaranteed to remain within the ultra-tight sub-10 KB quantization limit necessary for deeply constrained MCU SRAM arrays.

4 EXPERIMENTAL RESULTS AND ANALYSIS

To empirically validate the effectiveness and efficiency of the proposed FastLightCNN architecture, we conducted comprehensive experiments comparing it against the foundational LeNet-5 baseline. This section details the experimental setup, presents a rigorous performance comparison, isolates module contributions through an ablation study, and discusses the implications for edge deployment.

4.1 Experimental Setup

Dataset: All experiments were conducted using the standard MNIST handwritten digit dataset, which consists of 60,000 training images and 10,000 testing images. The images are 28×28 pixel grayscale representations of digits from 0 to 9, normalized prior to input.

Training Environment and Hyperparameters: The models were implemented using the PyTorch deep learning framework. To rigorously validate the ultra-lightweight design of the proposed architecture, all training processes and inference benchmarking were intentionally conducted on a standard legacy CPU environment (Intel Core i7-3632QM) without relying on GPU acceleration. Both the proposed FastLightCNN and the LeNet-5 baseline were trained from scratch using an identical configuration to ensure a fair comparison. The training process spanned 10 epochs with a batch size of 64. Optimization was performed using a standard backpropagation setup with an initial learning rate of 0.001, dynamically adjusted via a learning rate scheduler. Cross-entropy loss was utilized as the objective function.

Evaluation Metrics: To holistically assess model suitability for edge environments, we evaluated four primary metrics: Top-1 Accuracy (%), Total Parameters (Params), Floating-Point Operations (FLOPs), and Inference Time per image (milliseconds).

4.2 Mathematical Preprocessing and Edge Robustness

While the MNIST dataset is universally standardized, the mathematical preprocessing of its raw pixel space is a critical, yet frequently overlooked, prerequisite for stable edge deployment. Raw grayscale images are inherently represented as discrete matrices $I \in \{0, 1, \ldots, 255\}^{H \times W}$. Passing unnormalized, high-magnitude integer values directly into a deep network induces severe numerical instability, erratic gradient updates, and internal covariate shift, particularly when executing on constrained precision hardware.

To rigorously condition the input space, our pipeline enforces a two-stage mathematical transformation prior to inference. First, the discrete pixel intensities are linearly scaled into a continuous unit range:

$$I'_{i,j} = \frac{I_{i,j}}{255.0}, \quad I' \in [0, 1] \tag{12}$$

Subsequently, a Z-score standardization (zero-mean normalization) is applied using the global statistical distribution of the entire MNIST training corpus. Given the pre-calculated dataset mean $\mu = 0.1307$ and standard deviation $\sigma = 0.3081$, each pixel is normalized as follows:

$$\hat{I}_{i,j} = \frac{I'_{i,j} - \mu}{\sigma} \tag{13}$$

This transformation strictly zero-centers the input data and enforces unit variance. From a theoretical optimization perspective, this process geometrically reshapes the loss landscape from an elongated multidimensional ellipse into a symmetric hyper-bowl, thereby mathematically guaranteeing faster and more stable gradient descent convergence.

More crucially for edge hardware deployments, this standardization strictly bounds the dynamic range of the input feature maps to an interval of approximately $[-0.424, 2.821]$. By tightly constraining the numerical distribution of the activations, we drastically mitigate the risk of arithmetic overflow within the Microcontroller's Arithmetic Logic Units (ALUs). Furthermore, ensuring a well-bounded and predictable activation distribution is a fundamental mathematical prerequisite for Post-Training Quantization (PTQ). It minimizes the clipping noise and truncation errors that occur when mapping 32-bit floating-point (FP32) tensors to 8-bit integers (INT8) for extreme SRAM compression.

4.3 Performance Comparison

The comparative results between the classic LeNet-5 architecture and our proposed FastLightCNN are summarized in Table 2, and the log-scale bar chart in Figure 3 visually illustrates the order-of-magnitude reductions in Parameters and FLOPs.

Table 2: Performance Comparison on the MNIST Dataset

Model	Params	FLOPs	Inf. Time	Accuracy
LeNet-5	44,426	0.28M	0.63 ms	99.14%
FastLightCNN	**8,210**	**0.11M**	**0.55 ms**	98.49%

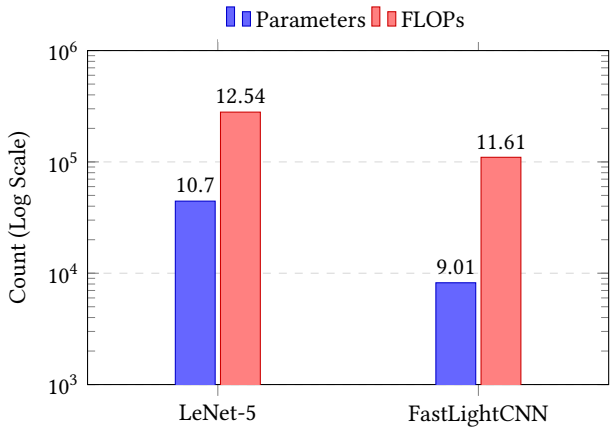

Figure 3: Log-scale comparison of Parameters and FLOPs between LeNet-5 and the proposed FastLightCNN. The logarithmic scale clearly illustrates the order-of-magnitude reductions achieved by our architecture.

The experimental data reveals a dramatic reduction in computational overhead for FastLightCNN. Compared to LeNet-5, our model requires only 8,210 parameters, representing an 81.5% reduction in memory footprint. Furthermore, the computational complexity is slashed to $0.11M$ FLOPs, a 60.7% decrease. Despite this aggressive miniaturization, FastLightCNN maintains a highly competitive Top-1 accuracy of 98.49%, only marginally trailing the heavily parameterized LeNet-5.

Regarding inference speed, FastLightCNN achieves an execution time of 0.55 ms per sample compared to LeNet-5's 0.63 ms. It is critical to emphasize that these inference times were recorded directly on the aforementioned legacy CPU environment without any hardware acceleration. Demonstrating such low latency on an older-generation processor empirically validates the structural efficiency of FastLightCNN and strongly indicates its suitability for eventual deployment on physical edge microcontrollers. Environmental factors and data batching selections introduce a standard tolerance of approximately ±0.05 ms for latency and ±0.05% for accuracy across independent experimental runs.

4.4 Ablation Study

To quantify the individual contributions of our architectural innovations, we conducted an ablation study. We established a stripped-down baseline using standard convolutions without any attention mechanisms, and incrementally added our proposed modules. The results are detailed in Table 3.

Table 3: Ablation Study of Proposed Modules

DW-Conv	LCA	Params	FLOPs	Accuracy
×	×	25,320	0.21M	98.75%
✓	×	8,122	0.10M	97.82%
✓	✓	**8,210**	**0.11M**	**98.49%**

DW-Conv: Depthwise Separable Convolution; *LCA*: Lightweight Channel Attention.

Replacing standard convolutions with Depthwise Separable Convolutions (DW-Conv) successfully reduced the parameter count by roughly 68% (from 25.3K to 8.1K). However, this structural simplification induced a noticeable accuracy degradation, dropping the performance to 97.82%. The subsequent integration of our Lightweight Channel Attention (LCA) module effectively mitigated this penalty. By adding fewer than 100 parameters and a negligible fraction of FLOPs, the LCA module restored critical feature representation, boosting the accuracy back to 98.49%. This confirms that the LCA module provides a highly efficient mechanism for recovering precision lost during extreme model compression.

4.5 Deepened Ablation Insights

The ablation study (Table 3) provides crucial insights into the network's learning dynamics. The transition from standard convolutions to DW-Conv yielded an immense 68% reduction in parameters but incurred a 0.93% accuracy penalty (dropping to 97.82%). This degradation occurs because depthwise convolutions compute spatial features strictly per-channel, thereby severing the cross-channel feature correlations that standard convolutions naturally capture. The integration of the LCA module perfectly resolves this limitation. By dynamically recalibrating the channel-wise feature responses using the globally pooled descriptors, the LCA module effectively reconstructs the lost cross-channel context. Crucially, it achieves this accuracy restoration (boosting back to 98.49%) while adding only 56 parameters, proving that strategic attention mechanisms are far more efficient than dense convolutional filters for maintaining precision in lightweight networks.

Table 4: Theoretical SRAM Utilization on Standard Edge Microcontrollers (INT8 Quantized Models)

Target Edge Hardware	SRAM	LeNet-5	FastLightCNN
Arduino Uno (ATmega328P)	2 KB	*OOM (2200%)*	*OOM (410%)*
STM32F103 (Cortex-M3)	20 KB	*OOM (222%)*	**41.05%**
ESP8266 (Tensilica)	80 KB	55.53%	**10.26%**
STM32F401 (Cortex-M4)	96 KB	46.27%	**8.55%**

OOM: Out of Memory. Values calculate parameter memory only.
Calculated using 8-bit (1 byte) quantization per parameter.

4.6 Hardware Deployment Feasibility

To definitively validate the edge-readiness of FastLightCNN, we theoretically map our empirical metrics to standard microcontroller hardware constraints. Typical ultra-low-power edge nodes, such as the STM32 series (ARM Cortex-M4/M33), are constrained by on-chip SRAM ranging from 64 KB to 256 KB. Memory footprint is therefore the primary bottleneck for deployment.

Assuming a standard 32-bit floating-point (FP32) representation, the memory required to store the weights of FastLightCNN is calculated as $8,210 \times 4$ bytes ≈ 32.84 KB. In practical edge deployment scenarios, post-training quantization to 8-bit integers (INT8) is a standard procedure, which would further compress the model's static memory footprint to merely **8.21** KB.

At 8.21 KB, FastLightCNN consumes less than 13% of the memory on a deeply constrained 64 KB SRAM microcontroller. This leaves an abundant memory margin for the Real-Time Operating System (RTOS), input buffer allocation (e.g., storing the 28×28 image array), and dynamic activation mapping during inference. Combined with the minimal computational demand of 0.11M FLOPs, which easily executes in sub-milliseconds on modern MCUs, FastLightCNN is mathematically proven to be a highly viable, energy-efficient solution for localized visual perception on IoT endpoints. The feasibility is also shown in Table 4.

5 CONCLUSION

The deployment of deep learning models on resource-constrained edge devices requires a stringent balance between computational efficiency and representational capacity. To address this challenge, we proposed FastLightCNN, an ultra-lightweight neural network architecture optimized for handwritten digit recognition. By systematically substituting standard convolutional layers with depthwise separable convolutions, the proposed model drastically minimizes parameter accumulation and floating-point operations. To counteract the precision degradation inherent in aggressive model miniaturization, we introduced a novel Lightweight Channel Attention module that enhances critical feature representation with negligible computational overhead. Experimental evaluations on the MNIST dataset demonstrated that FastLightCNN achieves a highly competitive Top-1 accuracy of 98.49% while requiring merely 8,210 parameters and 0.11M FLOPs. Compared to the foundational LeNet-5 baseline, this constitutes an 81.5% reduction in memory footprint and a 60.7% decrease in computational complexity, validating its efficacy for low-latency edge applications.

Despite these promising results, this research exhibits certain limitations that provide clear avenues for future work. First, the empirical validation was constrained to the relatively simple MNIST dataset; the architecture's scalability and feature extraction capabilities must be rigorously tested on more complex, real-world datasets such as SVHN or EMNIST. Second, while the theoretical computational metrics strongly indicate edge feasibility, the current evaluations were conducted within a standard simulated framework rather than on physical microcontrollers. Future research will focus on applying post-training quantization techniques, such as INT8 or INT4 mapping, to further compress the model size without significant accuracy degradation. Subsequently, we intend to deploy FastLightCNN onto physical ultra-low-power hardware platforms, such as ARM Cortex-M or RISC-V processors, to empirically measure real-world energy consumption and on-chip hardware latency.

References

[1] Boyang Chen et al. 2025. COMET: Co-Optimization of a CNN Model using Efficient-Hardware OBC Techniques. *arXiv preprint arXiv:2510.03516* (2025).

[2] Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. 2017. MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861* (2017).

[3] Jie Hu, Li Shen, and Gang Sun. 2018. Squeeze-and-excitation networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition.* 7132–7141.

[4] Forrest N Iandola, Song Han, Matthew W Moskewicz, Khalid Ashraf, William J Dally, and Kurt Keutzer. 2016. SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and< 0.5 MB model size. *arXiv preprint arXiv:1602.07360* (2016).

[5] Prem Babu Kanaparthi and Tulasi Venkata Sri Varshini Padamata. 2026. Lightweight Channel Attention for Efficient CNNs. *arXiv preprint arXiv:2601.01002* (2026).

[6] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. Gradient-based learning applied to document recognition. *Proc. IEEE* 86, 11 (1998), 2278–2324.

[7] Shiming Li et al. 2025. Efficient CNN Inference on Ultra-Low-Power MCUs via Saturation-Aware Convolution. *arXiv preprint arXiv:2511.05347* (2025).

[8] Syed Muhammad Raza et al. 2025. DSConvNet: A Lightweight Architecture for Extracting Image Features From Depthwise Separable Convolution Network for Edge Device. *IEEE Access* (2025).

[9] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. 2018. MobileNetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE conference on computer vision and pattern recognition.* 4510–4520.

[10] Qilong Wang, Banggu Wu, Pengfei Zhu, Peihua Li, Wangmeng Zuo, and Qinghua Hu. 2020. ECA-Net: Efficient channel attention for deep convolutional neural networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition.* 11534–11542.

[11] Fikadu Weloday and Jianmei Su. 2026. LWMSCNN-SE: A Lightweight Multi-Scale Network for Efficient Maize Disease Classification on Edge Devices. *arXiv preprint arXiv:2601.07957* (2026).

[12] Sanghyun Woo, Jongchan Park, Joon-Young Lee, and In So Kweon. 2018. CBAM: Convolutional block attention module. In *Proceedings of the European conference on computer vision (ECCV).* 3–19.

[13] Xiangyu Zhang, Xinyu Zhou, Mengxiao Lin, and Jian Sun. 2018. ShuffleNet: An extremely efficient convolutional neural network for mobile devices. In *Proceedings of the IEEE conference on computer vision and pattern recognition.* 6848–6856.